

WORKSHOP · JUNE 2026

From Idea → PR

What changes when agents do the build work.

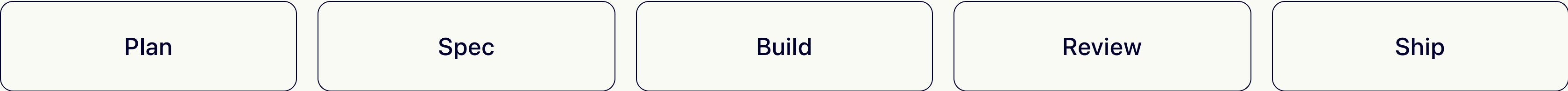
RUNTIME

3h-4h

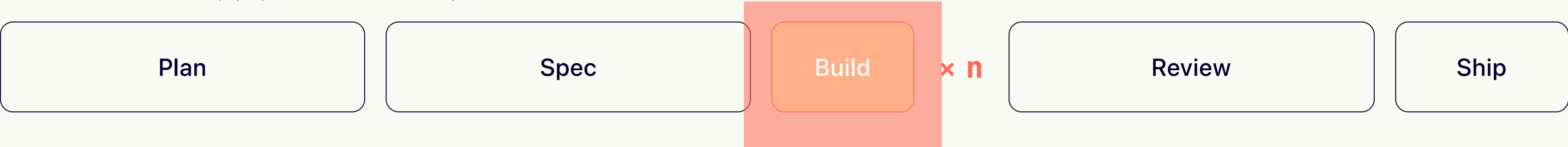
The build phase is shrinking.

Build compresses. Plan and review expand to fill the space.

OLD SHAPE



NEW SHAPE smaller loops, repeated n times before ship



Calibration.

Quick raise of hands.
Where are you today?

-
- 01 AGENTS.md committed in your repo?
 - 02 Your own custom skills running?
 - 03 Plan mode used recently?
 - 04 Any SDD framework tried?
 - 05 Where did it create friction with how you work today?
 - 06 Matt Pocock's grill-with-docs — who's looked?
-

You've hired a
new colleague.

Capable. New. No context.

No AGENTS.md yet? Bootstrap one.

Step 1 of 2: paste this in the agent. Zero shell commands — works identically in any harness.

PROMPT

Analyze this codebase and create an AGENTS.md file:

1. Keep it under 150 lines (measured optimum: 100–150)
2. Cover: WHAT (tech stack), WHY (purpose), HOW (commands)
3. Progressive disclosure: index pointing to docs/ files with a one-line description of each, so they're only read when relevant
4. file:line references instead of code snippets
5. Document the 2–3 most critical workflows (build, test, release) as numbered step-by-step lists
6. Assume linters handle code style – no formatting rules
7. Always include these two lines:
 - Be extremely concise. Sacrifice grammar for concision.
 - At the end of each plan, list unresolved questions.

Extract recurring patterns into docs/architectural_patterns.md
Extract procedural know-how into skills under
.agents/skills/<name>/SKILL.md (YAML frontmatter with
name + description, instructions below)

WHAT YOU GET

- Root file 100–150 lines
- WHAT · WHY · HOW only
- Index into docs/ — read when relevant
- 2–3 critical workflows as numbered steps
- Skills under .agents/skills/

Same prompt in Claude Code, Codex, Cursor, Copilot — no permissions or approval mode in play.

Treat the output as a first draft. Read it, delete what's obvious, keep what's load-bearing.

Step 2: ten seconds of wiring. **The harness tax.**

Claude Code reads `CLAUDE.md` and `.claude/skills` — not the standard. Deterministic, so it runs in the terminal, not the agent.

MACOS / LINUX

```
# Claude Code reads CLAUDE.md, not AGENTS.md
echo "@AGENTS.md" > CLAUDE.md

# Claude Code reads .claude/skills, not .agents/skills
mkdir -p .claude && ln -s ../agents/skills .claude/skills
```

WINDOWS (POWERSHELL)

```
Set-Content CLAUDE.md "@AGENTS.md"
New-Item -ItemType Directory -Force .claude | Out-Null
Copy-Item -Recurse .agents\skills .claude\skills
```

WHY A SEPARATE STEP

- Nothing an agent can interpret creatively
- Repo stays on the standard: `AGENTS.md` + `.agents/skills`
- Tax expires when native support lands — issue #6235, 5,200+ reactions

Windows caveat: copies instead of links — re-sync `.claude/skills` after editing a skill.

The handbook the agent reads first.

AGENTS.md is small on purpose.

THREE THINGS AT THE ROOT

- One-sentence project description
- Package manager, if not npm
- Non-standard build / typecheck commands

Everything else lives in docs/ files and skills.

IF YOURS ALREADY GREW BIG

Refactor my AGENTS.md to follow progressive disclosure.

1. Find contradictions. Ask which to keep.
2. Extract essentials for the root: one-line description, package manager, build / typecheck, anything truly relevant to every task.
3. Keep the 2–3 most critical workflows in the root as numbered steps – they earn their place.
4. Group the rest into categories. One markdown file per category in docs/. Move procedural know-how into .agents/skills/<name>/SKILL.md.
5. Output a minimal root AGENTS.md linking each file with a one-line description. Replace code snippets with file:line refs.
6. Flag for deletion: redundant, vague, or overly obvious instructions.

The agent's desk: the context window.

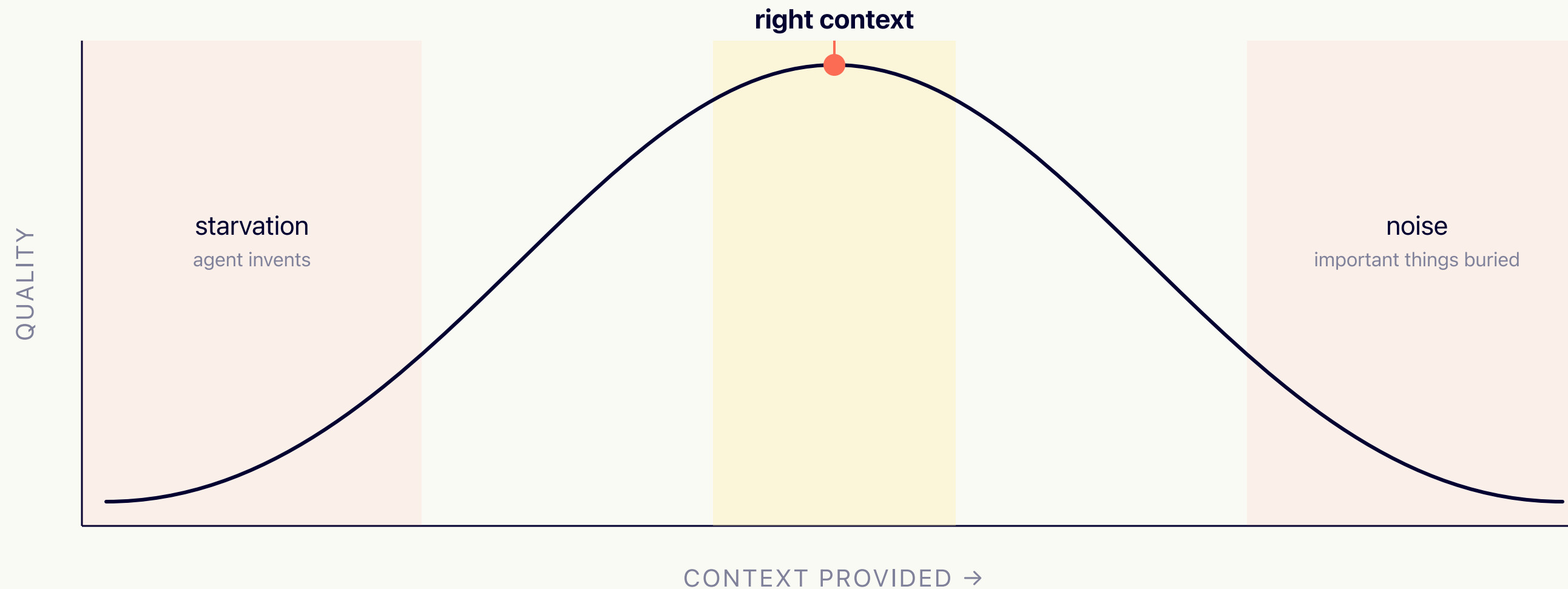
WHAT'S ON THE DESK

- System prompt
- AGENTS.md / team rules
- Conversation
- Issue / spec
- Files the agent has read
- Tool output
- Test failures
- Review comments

If it's not on the desk,
it doesn't exist.

Too little, too much, or right.

More context isn't better. **Better context** is better.



Same benchmark. Wildly different models.

Fable 5 just shipped (9 June 2026) — Mythos-class, 1M window — but with no comparable MRCR score published. Among the models that do report, performance diverges wildly at 1M tokens — **pick the model for your context budget, and pin it.**

Accuracy on MRCR v2 8-needle, by context length

Claude Fable 5

Anthropic · Mythos-class · 9 June 2026

Claude Opus 4.8

Anthropic · best published MRCR · Fable's fallback

GPT-5.5

OpenAI · current

GPT-5.2

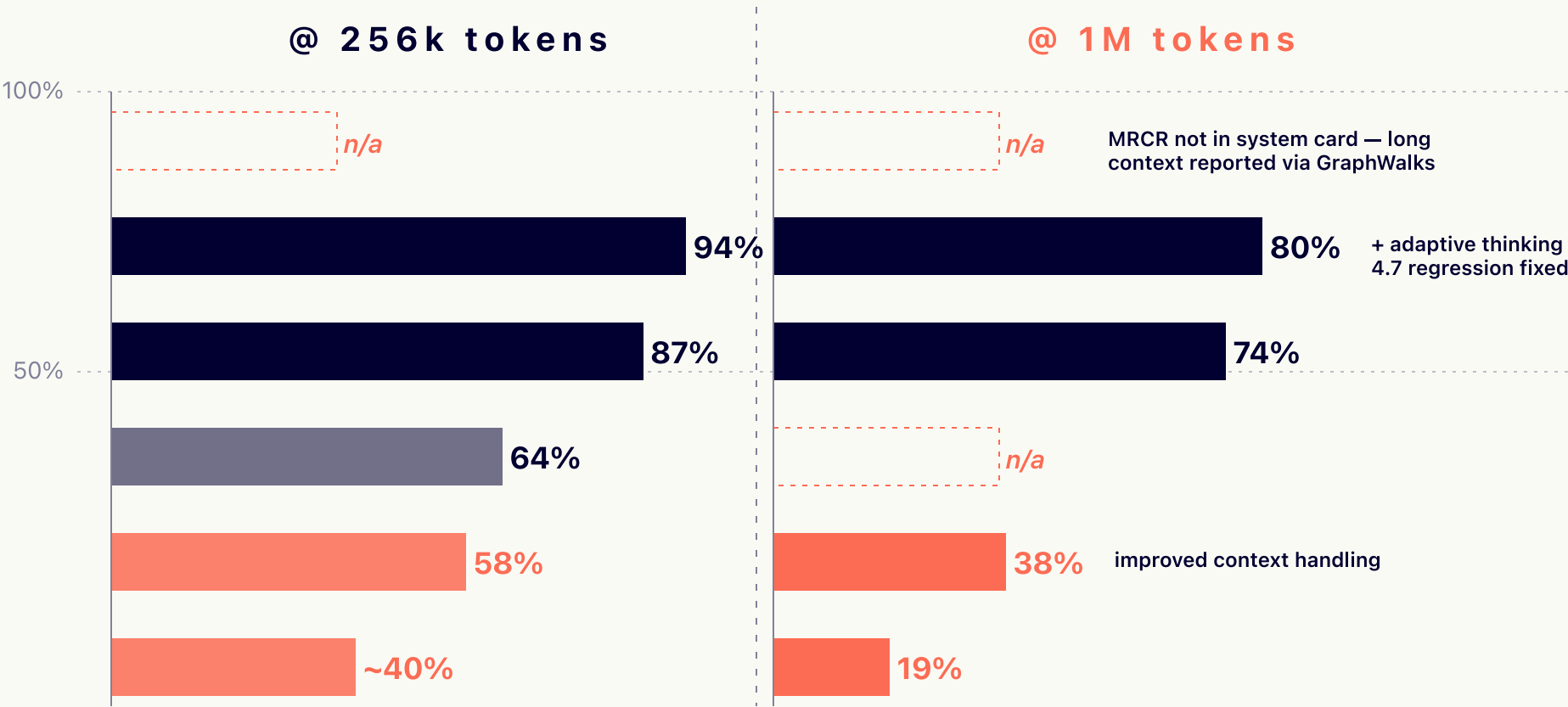
OpenAI · two gens back

Gemini 3.5 Pro

Google · current as of June 2026

Claude Sonnet 4.5

Anthropic · two gens back



Source: MRCR v2 (8-needle), via Context Arena, llm-stats.com and vendor system cards, retrieved 11 June 2026. Fable 5 / Mythos 5 (9 June) report long context via GraphWalks (91.1 F1 BFS @256k, 79.4 @1M — ahead of Opus 4.8), not MRCR — shown as n/a. Original benchmark: Google DeepMind (Michelangelo, arXiv 2409.12640). Some 256k figures interpolated. Treat as model-comparison signal, not absolute accuracy.

Open source for the **execute** step.

Keep the frontier model for planning. For implementation, open-weights models now **match or beat the frontier of six months ago** — at 80–95% lower cost per session.

Headline coding score · open-weights models

DeepSeek V4-Pro

MIT · API \$0.50/\$1.50 · best all-rounder, executor

Qwen 3.6

Apache 2.0 · 1M native context · best balance

Llama 4 Maverick

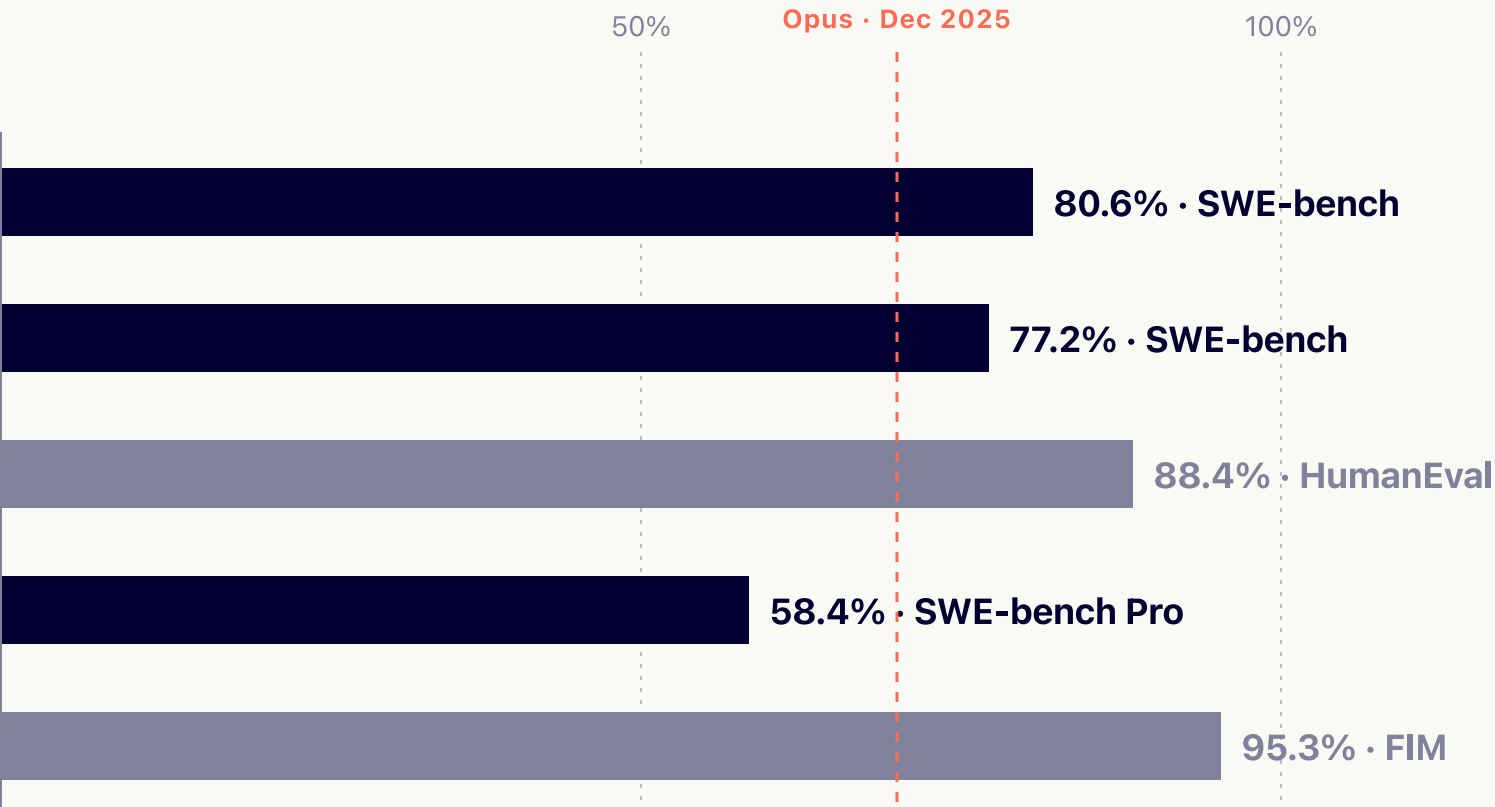
open weights · on-prem · Western orgs, no China licensing

GLM-5.1

MIT · frontier reasoning · hardest benchmark here

Mistral Codestral 22B

fast · 256K context · inline autocomplete only



Different benchmarks (SWE-bench / SWE-bench Pro / HumanEval / FIM) — **not directly comparable**; read each as a per-model signal, not a ranking. Navy = SWE-bench family, grey = other tasks. Dashed line = Claude Opus's SWE-bench level in Dec 2025 (≈70%) — the leap since. Pricing per M tokens (API) where shown. As of June 2026 — refresh as releases land.

Tokenomics: same eval, wildly different token bills.

Price per million is half the invoice. The other half: how many tokens a model burns to produce the output. Same 10-eval suite, up to a 13x spread — budget tokens per task, not dollars per million.

Output tokens generated to run the full Intelligence Index (10 evals, same tasks for every model)

Claude Opus 4.6 (max)

Anthropic · two gens back · scores 53

GPT-5.4 (xhigh)

OpenAI · one gen back · scores 57

Claude Opus 4.8 (max)

Anthropic · current · scores 61

GPT-5.5 (xhigh)

OpenAI · current · scores 60

GPT-5.5 (high)

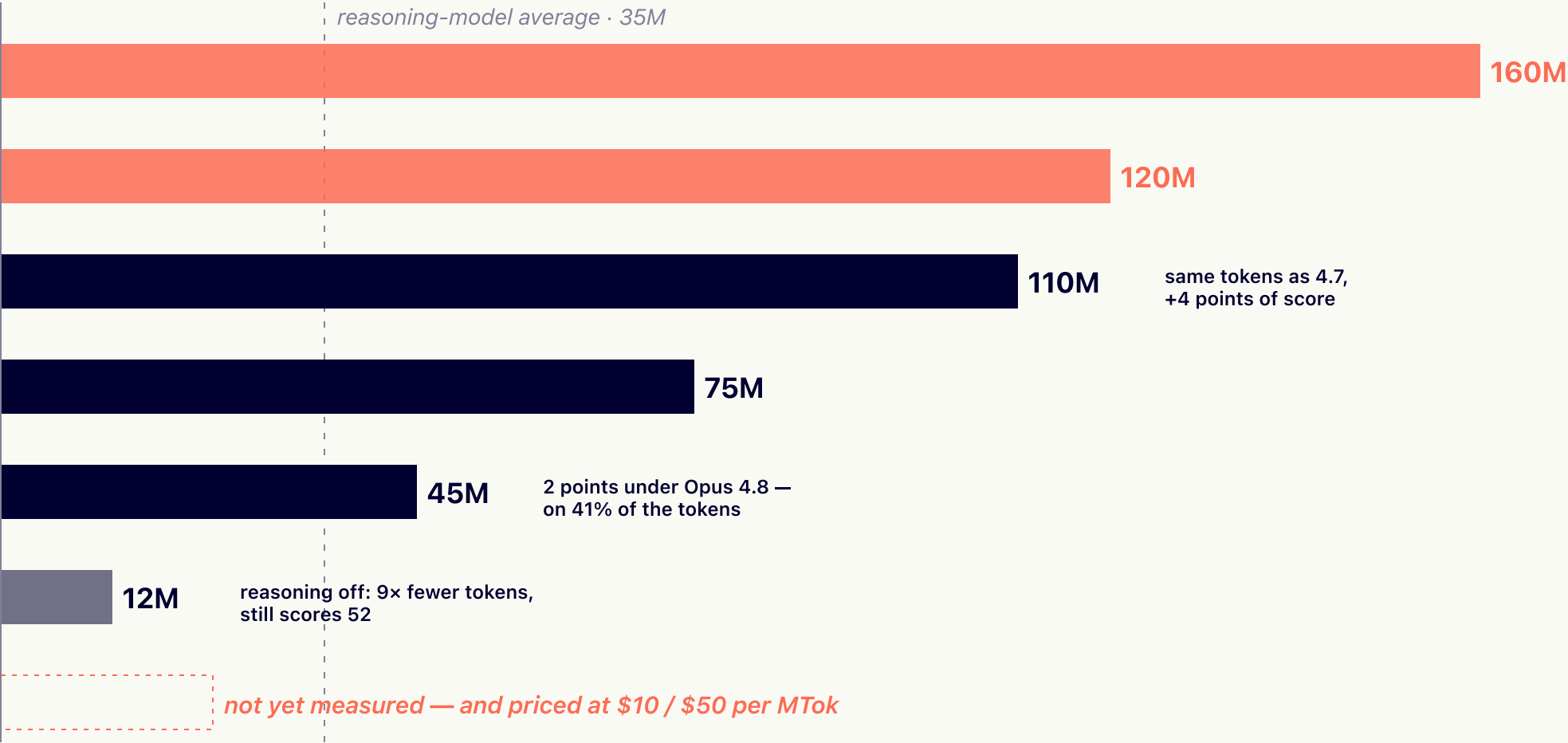
OpenAI · current · scores 59

Claude Opus 4.7 (no reasoning)

Anthropic · reasoning off · scores 52

Claude Fable 5

Anthropic · Mythos-class · scores 65 (#1)



Source: Artificial Analysis Intelligence Index v4 (10 evals incl. GDPval-AA, Terminal-Bench Hard, GPQA Diamond), model pages retrieved 11 June 2026. Bars = output tokens generated across the full suite; reasoning-model average ≈ 35M, non-reasoning average ≈ 8M. Fable 5 verbosity not yet published. Treat as verbosity signal — cost per task = price × tokens.

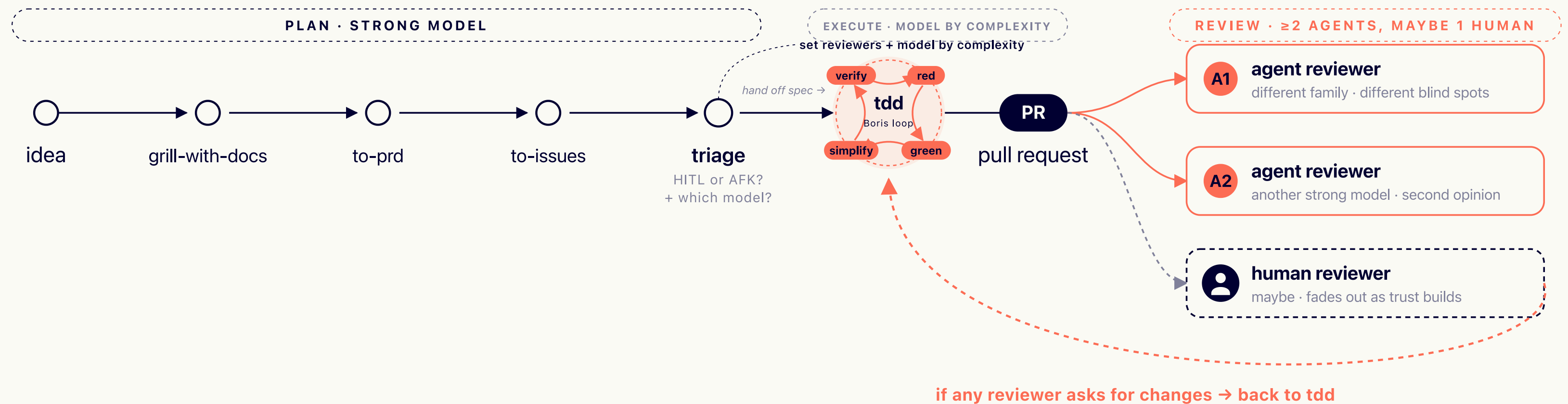
Old flow: alignment was front-loaded.

chat → docs → issues → plan → **long build** → PR review

Not wrong. Optimised for slow build.

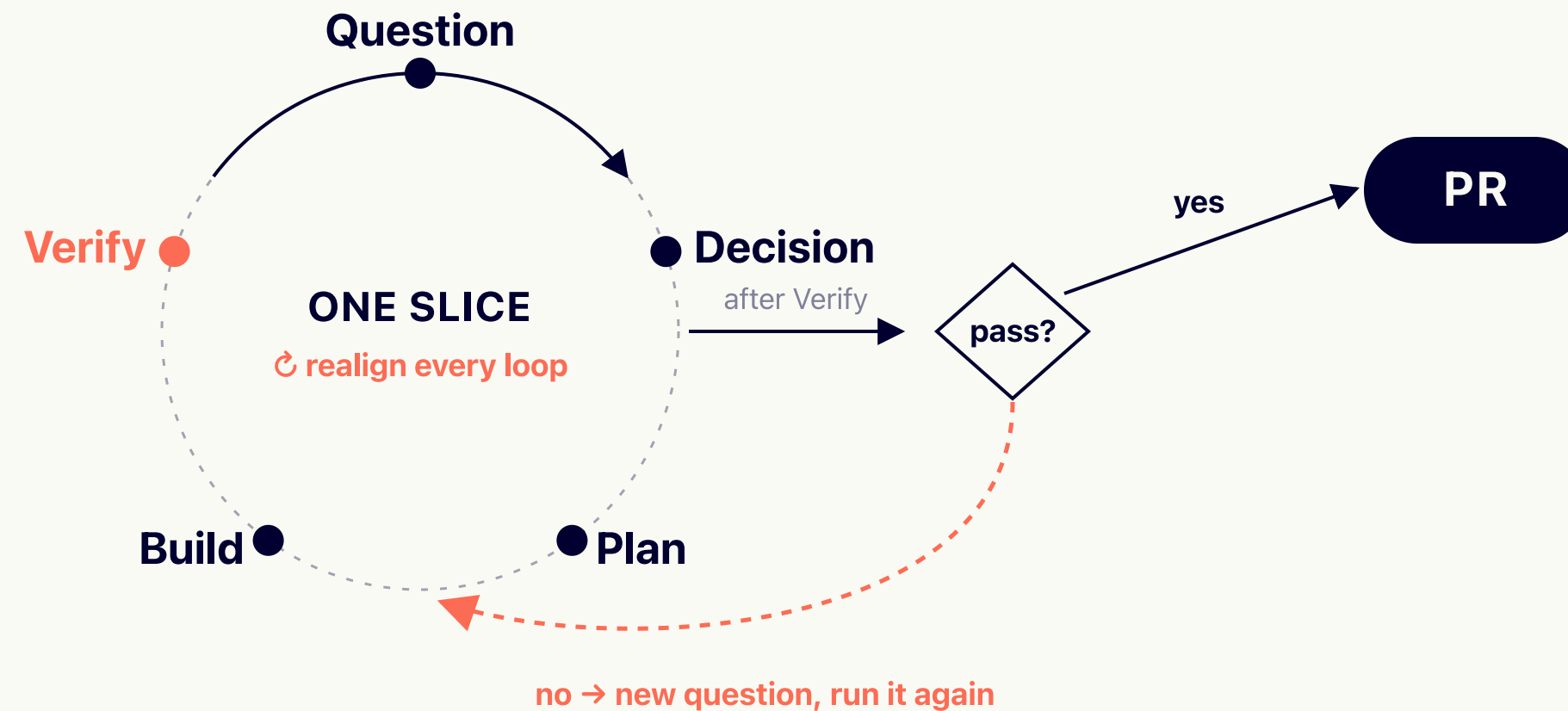
From idea to PR.

Plan with a strong model. **Triage routes each slice by complexity** — a cheaper model for simple work, a stronger one when it's hard. Then review on a different model again, maybe a human until you trust the loop.



New flow: repeated alignment.

Question → Decision → Plan → Build → Verify. Repeat.



One slice runs the whole loop. Because Question and Decision come back every pass, drift is caught inside the loop — it only becomes a PR once Verify is clean.

Get the context **out of the team's heads.**

Most of what the agent needs is tacit — it lives in the team, not the repo.

BEFORE

It only came out at the keyboard.

- Breakdown covered the obvious; the rest stayed in someone's head.
- It surfaced while a dev was coding — late, and trapped in one person.
- Misalignment showed up at PR review, when changing it was expensive.

NOW

The agent pulls it out in breakdown.

- Reads the repo, then asks one question at a time until the team's knowledge is on the page.
- Each issue comes out smaller — a slice sized to fit one context window.
- Smaller slices matter most when the executor is an open-source agent.

Smaller, explicit slices = context you can actually fit — and what makes open-source executors viable.

Break it down with the agent.

Read a feature.

It reads the code. That changes the conversation.

This is spec-driven development — but flexible.

Same DDD / BDD / TDD muscle.

Independently invocable steps.
No locked pipeline.

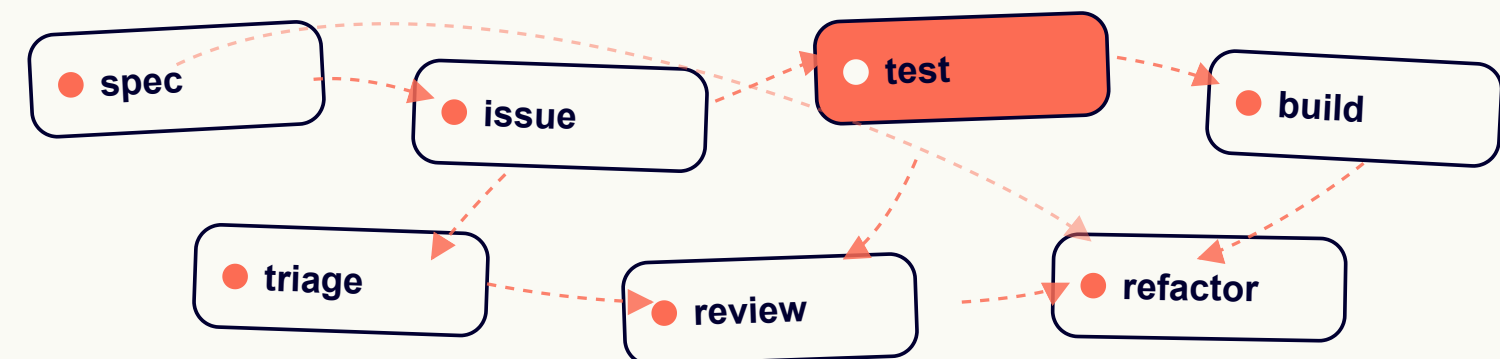
CLASSIC SDD · LOCKED PIPELINE



spec → issue → test → build → done. in that order.

BREAK THE CHAIN

SAME PARTS · PICK ANY, IN ANY ORDER



invoke whichever, whenever · skip what you don't need

`/setup-matt-pocock-skills` , `/grill-with-docs` , `/to-prd` , `/to-issues` , `/triage` , `/tdd` ,
`/improve-codebase-architecture` , `/zoom-out` , `/diagnose` (*ideation: `claude.ai/design` + `/handoff` `/prototype`*)

Three letters you already know.

The agent doesn't replace this craft. **It runs on it.**

DDD

DOMAIN-DRIVEN DESIGN

Name the concepts
before you code them.
Shared language between
PO and dev.

*Eric Evans · Domain-Driven Design
· 2003*

Lives in **grill-with-docs** &
CONTEXT.md.

BDD

BEHAVIOUR-DRIVEN
DEVELOPMENT

Describe behaviour in
given / when / then.
Acceptance before
implementation.

Dan North · Introducing BDD · 2006

Lives in **to-prd** & **to-issues**.

TDD

TEST-DRIVEN DEVELOPMENT

Red → green → refactor.
One slice, one diff.
Test is the contract the
agent meets.

*Kent Beck · Test-Driven Development:
By Example · 2003*

Lives in **tdd**.

Install once — but **where?** Scope is a file path.

One workflow, any harness — the open `.agents/` layout. The directory decides who gets the skill: **where SKILL.md lives is where it works.**

ONE REPO · THE TEAM

```
acme-app/
├── AGENTS.md
└── .agents/skills/
    └── tdd/SKILL.md
```

```
npx skills add mattpocock/skills
```

Committed to git — every teammate, every harness, and CI get it at checkout.

MONOREPO · ALL PACKAGES

```
monorepo/
├── AGENTS.md
├── .agents/skills/ ◀ root
│   └── tdd/SKILL.md
├── packages/api/
└── packages/web/
```

```
cd monorepo && npx skills add ...
```

One copy at the checkout root — skills are read upward from wherever the agent starts, so `packages/web` needs no copy of its own.

ALL REPOS · JUST YOU

```
~/.agents/skills/
└── tdd/SKILL.md

symlinked per harness:
~/.claude/skills → ...
```

```
npx skills add mattpocock/skills -g
```

Personal: every project on this machine. Doesn't sync, teammates never see it.

ALL REPOS · WHOLE TEAM

```
org/skills-repo
├── skills/
│   └── tdd/SKILL.md
└── .claude-plugin/
    └── plugin.json ◀ optional
```

```
npx skills add org/skills-repo
```

A versioned skills repo — one command in any harness. The optional manifest also makes it a namespaced Claude Code plugin.

`.agents/` is the cross-harness standard — the skills CLI symlinks it into Claude Code, Codex, Cursor, Copilot + 60 more (Claude wiring: the harness-tax step from earlier). Same name at several levels: enterprise overrides personal overrides project. Start committed-in-repo; graduate to a skills repo when a second repo wants the same skills.

SKILL · ONCE PER MACHINE, ONCE PER REPO

/setup-matt-pocock-skills

Without it, every other skill runs in degraded mode.

ONCE PER MACHINE

```
npx skills@latest add mattpocock/skills
```

ONCE PER REPO

```
/setup-matt-pocock-skills
```

SKILL · THE LOAD-BEARING ONE

/grill-with-docs

Sprint-planning equivalent for agentic work.
Surface assumptions before code.

CONTEXT.md THE SHARED GLOSSARY

Domain terms only — zero implementation detail. Created lazily: the first resolved term starts the file. “Your glossary says X, you seem to mean Y” is the skill doing its job.

After a few runs: you say “the materialization cascade”, not three sentences — every session opens already speaking your language, and drift gets called out.

docs/adr/0001-...md THE WHY BEHIND DECISIONS

One file per non-obvious decision — trade-off, alternatives, reasons. Only created when all three hold: hard to reverse · surprising without context · a real trade-off.

After a few runs: the agent stops re-proposing ideas you already rejected — decided once, recorded, respected.

Both live in git, reviewable like code — and the other skills read them before acting: grill cross-checks, improve-codebase-architecture mines them, handoff links them instead of re-explaining. The repo's memory between sessions.

IN

- One real idea / ticket / customer need
- Existing CONTEXT.md (or empty)
- Repo access

OUT

- Sharpened problem statement
- Updated CONTEXT.md / ADRs
- Decision: ready, or not

SKILL · BDD LIVES HERE

/to-prd

Conversation → PRD → tracker issue.
The grilled idea becomes a thing the rest of the chain can act on.

A PRD ANSWERS FOUR THINGS

What should change	For whom	Why now	What evidence proves it shipped
--------------------	----------	---------	---------------------------------

SKILL · THE SLICING ONE

/to-issues

PRD → vertical slices, independently grabbable.

Marks blocking relationships — what runs in parallel, what waits.

GOOD SLICE

Internal user creates X, saves it, sees it in Y, behind flag Z.

BAD SLICE

~~Wire up the API endpoint.~~

OUTPUT: A DAG, NOT A LIST

Each issue tagged **HITL** (needs a decision) or **AFK** (agent-grabbable). Blocking edges drawn explicitly — independent slices ship in parallel, blocked ones wait their turn.

SKILL · THE CONTRACT

/triage

The gate from backlog to agent-ready.
Each issue earns one tag. The tag is the contract.

STATE MACHINE



Categories: bug · enhancement

SKILL · THIS IS WHERE THE DIFF COMES FROM

/tdd

Red → green → refactor. One slice at a time.



RED

Failing test against the agent-ready acceptance criteria.



GREEN

Smallest code change to pass — only inside the named boundaries.



REFACTOR

Simplify. Follow local patterns. Remove anything you can't defend.

One run = one diff = one PR. Don't pile slices.

SKILL · BETWEEN SLICES, AFTER MERGE

/improve-codebase-architecture

Find deepening opportunities — informed by your docs.
Regular cadence. Not part of the per-slice loop.

SKILL · ESCAPE HATCH, MID-TDD

/zoom-out

When the agent is stuck on unfamiliar code.
Step back, get the broader context, resume.

SKILL · DISCIPLINED DIAGNOSIS LOOP

/diagnose

reproduce → minimise → hypothesise → instrument → fix → regression-test

For hard bugs and performance regressions where the cause is not obvious.

Sprint breakdown — one of your issues.

PHASE A · DEMO	On the demo repo (populated CONTEXT.md + issue tracker). <code>/grill-with-docs</code> end-to-end · stops before <code>/tdd</code>
PHASE B · ADAPT	Live skill-adaptation on the driver's laptop. <code>/setup-matt-pocock-skills</code> → your issue tracker + git host. ~2 min.
PHASE C · WALKTHROUGH	One real ticket from your backlog. The room is the sprint-breakdown team. <code>/grill-with-docs</code> → <code>/to-prd</code> → <code>/to-issues</code> → <code>/triage</code> → <code>/tdd</code>

POs answer product questions. **Devs** answer code questions. Per facilitates — between the agent's questions, not while it's working.

From here: GitHub or local.

Same backlog. Two surfaces. Pick the one that fits the slice.

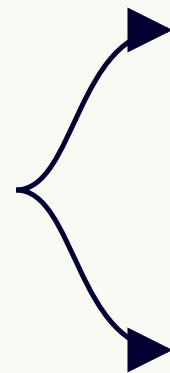
YOUR BACKLOG

● PROJ-401 slice 1 · ready-for-agent

● PROJ-402 slice 2 · ready-for-agent

● PROJ-403 slice 3 · ready-for-agent

● PROJ-404 slice 4 · ready-for-agent



GITHUB cloud agents pick from the queue

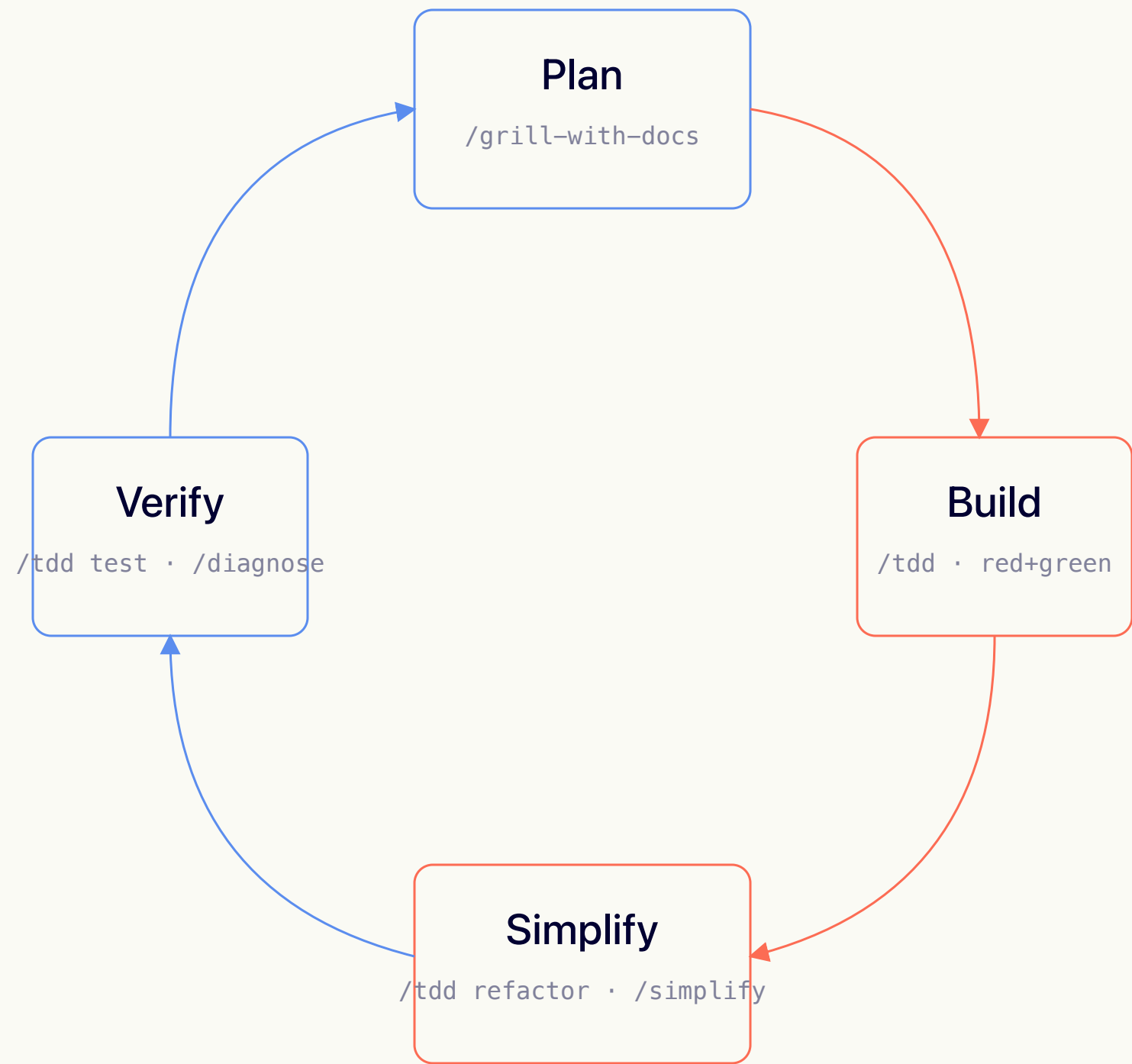
Assign an issue to a cloud agent. It works in a remote sandbox, opens a draft PR, posts back. Good for parallelisable, low-context slices.

LOCAL sandcastle on your machine

Pull issues into local worktrees. You see every step, intervene at any point. Good for high-context slices, refactors, anything tricky.

One slice, one loop.

Plan → Build → Simplify → Verify.



MAPPING

Plan	/grill-with-docs	interview plan; cite CONTEXT.md / ADRs
Build	/tdd	red → green; smallest code change
Simplify	/tdd refactor	extract dupes, deepen modules
	/simplify	review the diff for reuse, quality
Verify	/tdd test	run the suite; verify behaviour
	/diagnose	reproduce → minimise → fix

HORIZON

Sandcastle: loops in parallel.

TODAY

One slice, one loop, one human watching.

TOMORROW

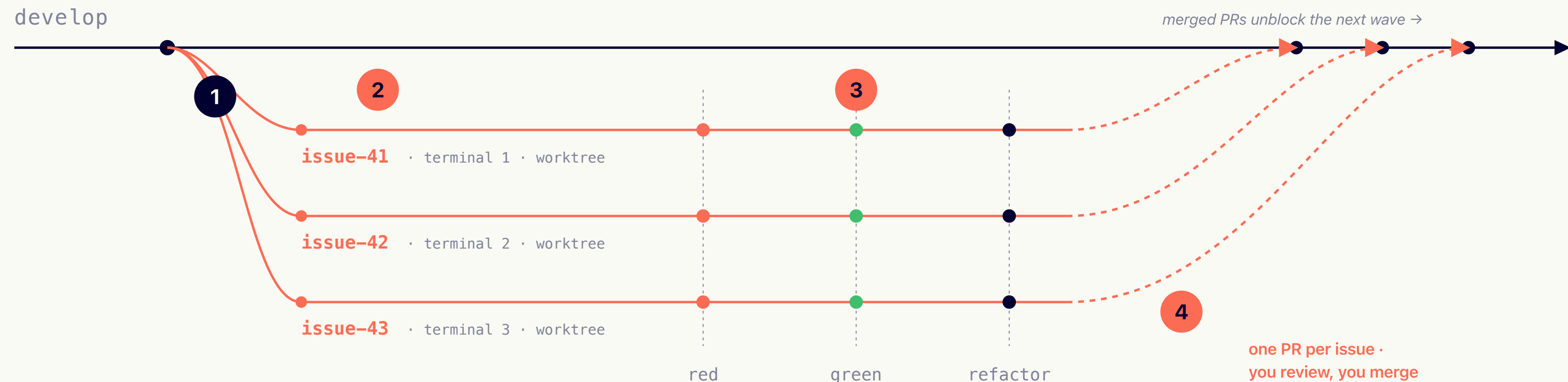
Many loops, in parallel, locally — same human, fewer keystrokes.

EVERY PR

Created — not merged. 2 LLM reviewers + 1 human, until you trust the LLM.

No sandcastle? Same flow, in terminals.

One terminal per unblocked issue, /tdd in each. You are the planner, the reviewer, and the merger.



1 You are the planner

Pick unblocked ready-for-agent issues from triage. One terminal per issue — two or three is plenty.

2 One worktree per terminal

Own branch, own filesystem, fresh context — the same isolation rule, hand-rolled. Parallel sessions never collide.

3 /tdd in each

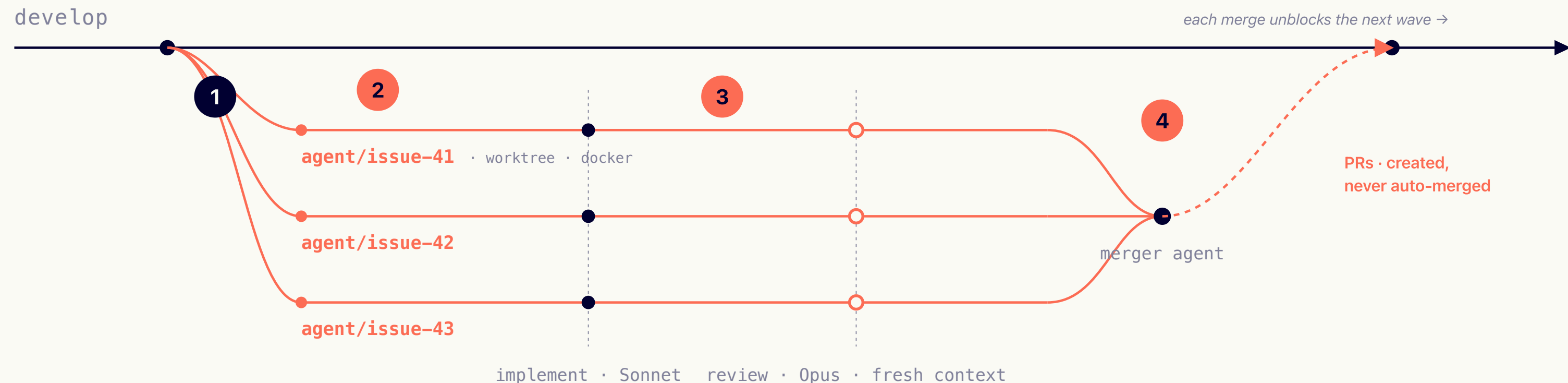
Red → green → refactor against the issue's acceptance criteria. One run = one diff. Stalls? /zoom-out, then resume.

4 You review, you merge

One PR per issue. Each merge unblocks the next wave — open the next terminals. Your review bandwidth is the bottleneck.

How sandcastle fits gitflow.

1 issue = 1 worktree = 1 branch = 1 fresh context. You come back to **a pile of PRs, not unfinished work.**



1 Planner picks unblocked issues

to-issues cut vertical slices with blocking relations, classed HITL or AFK. The planner takes what's ready.

2 One of everything per issue

Worktree, branch, Docker sandbox, fresh context. Context isolation, not just file isolation — each issue sized for the smart zone.

3 Build ≠ review

Sonnet implements, Opus reviews — and they never share a context. A reviewer in the dumb zone misses what a fresh one catches.

4 Merge unblocks the next wave

A merger agent lands the branches; the planner picks again. You read the PRs — “the code is your battleground.”

And the PR pile? Cloudflare put review agents in CI.

AI review in the critical path of CI — overridden 0.6% of the time. The human stays the escape hatch.

5,169

REPOSITORIES

131,246

REVIEWS · 30 DAYS

3m 39s

MEDIAN REVIEW

\$0.98

MEDIAN COST

0.6%

BREAK GLASS

ONE COORDINATOR · SEVEN SCOPED REVIEWERS

- Coordinator (top-tier model) spawns specialists — security, performance, quality, docs, release, compliance, AGENTS.md — then judges them into one structured comment.
- The prompt value lives in what **NOT** to flag — nitpicks teach developers to ignore the bot. 1.2 findings per review, deliberately low.
- Bias toward approval: warnings ship with comments, critical & security block, a human break glass comment overrides everything.

FOUR LESSONS FOR THE ROOM

- **Own the harness** — open-source agent + real SDK, not a rented closed tool.
- **Specialise and constrain** — many scoped agents beat one big prompt; structured severity drives behaviour.
- **Scale compute to risk** — a typo fix gets 2 agents at \$0.20, an auth/ path gets all 7+ at \$1.68.
- **Engineer for failure** — circuit breakers, fallback chains, a five-second kill switch.

Source: "Orchestrating AI Code Review at scale", Ryan Skidmore, Cloudflare blog · Agents Week, 20 April 2026 — blog.cloudflare.com/ai-code-review. Their own caveat: not a replacement for human review — a filter that removes the wait.

THREE TAKEAWAYS

What to carry out of the room.

- 01 DDD / TDD / BDD experience transfers directly.
- 02 What used to live in the code now lives in the process.
- 03 Programming the programmer is the new engineering.

ONE LAST LOOP

An agent without eval drifts.

So does a presenter.

Drop one signal — what worked, what didn't,
what I'll rewire before next round.

```
$ /eval-the-workshop --honest
```

~ 2 MIN · ANONYMOUS

THANKS FOR THE ROOM.

Stay in the loop.

CONTACT

Per Hassle

Monterro · InfuseAI

per.hassle@monterro.com

DISCORD · THE COMMUNITY

Where the loops keep going after today.

discord.gg/Mu2CrWFBQ7

THE SIGNAL · NEWSLETTER

What we ship, read, and rewire next.

monterro.com/why-us/infuse-ai/the-signal

DISCORD



THE SIGNAL

